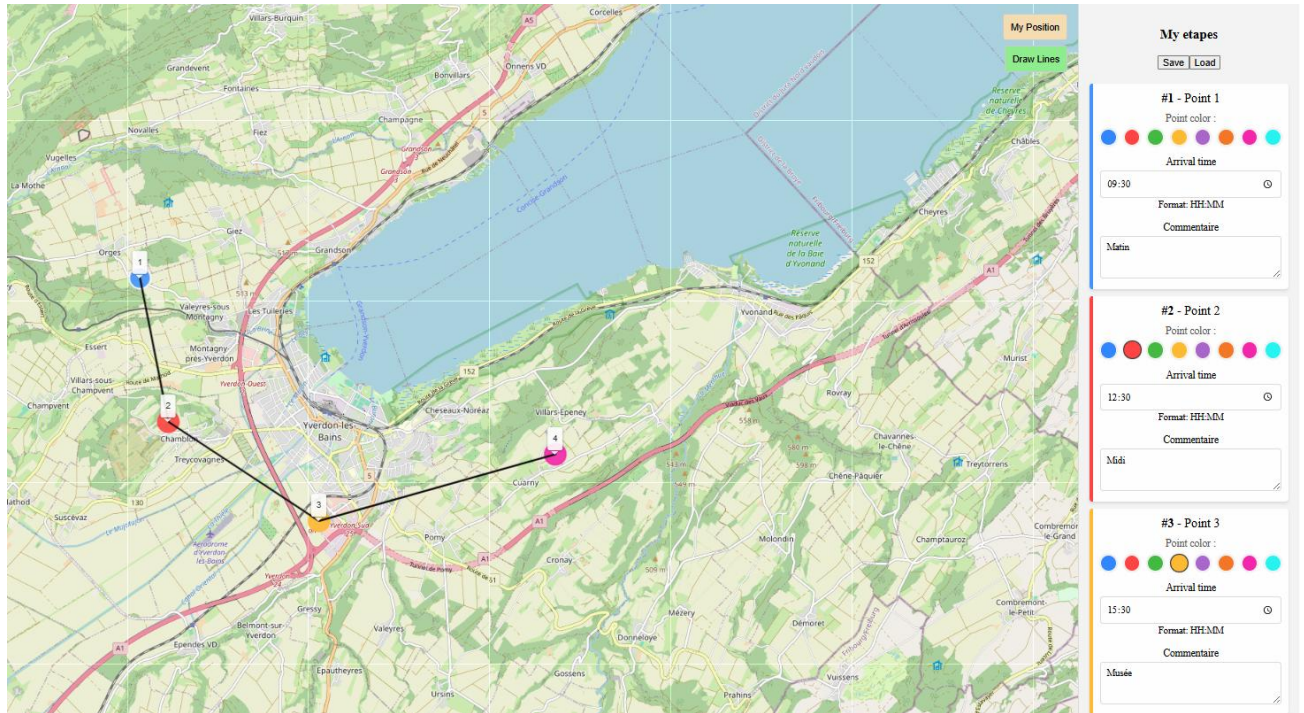


Mon Parcours



Projet Tpi - 2026

Ahmet Karabulut SI-CA2a

Table des matières

1	Analyse préliminaire	5
1.1	Introduction	5
1.2	Glossaire	5
1.3	Objectifs	6
1.3.1	Objectifs techniques	6
1.3.2	Objectifs fonctionnels	6
1.3.3	Objectifs pédagogiques	6
1.4	Planification initiale	7
1.4.1	Tableau récapitulatif	7
1.4.2	Détail des activités	8
1.4.3	Détail des Sprints	8
2	Analyse / Conception	11
2.1	Technologies utilisées	11
2.2	Outils d'aide	11
2.3	Modèle Conceptuel de Données (MCD)	12
2.4	Modèle Logique de Données (MLD)	13
2.5	Maquettes	14
2.6	Stratégie de test	18
2.6.1	Choix de l'outillage : Pourquoi Vitest ?	18
2.6.2	Configuration du projet	18
2.6.3	Structure et Co-location (__tests__)	19
2.7	Risques techniques	20
2.7.1	Identification	20
2.7.2	Analyse (Matrice des risques)	20
2.7.3	Évaluation et Priorisation	20
2.7.4	Traitement (Stratégies de réduction)	20
2.7.5	Suivi et contrôle	21
2.8	Planification	21
2.9	Dossier de conception	22
2.9.1	Architecture modulaire des composants visuels	22
2.9.2	Routage (Logique de l'itinéraire)	22

2.9.3	Gestion des appels de données (OpenStreetMap & Geolocation).....	23
2.9.4	Gestion de l'Interface Utilisateur (UI/UX)	23
3	Réalisation.....	23
3.1	Dossier de réalisation	23
3.1.1	Structure du repository	24
3.2	Construction de la documentation	24
3.3	Résultat des tests effectués	25
3.3.1	Tests unitaires :.....	25
3.3.2	Tests manuels :	25
3.4	Couverture des tests.....	27
3.4.1	Méthodologie de test des composants	27
3.4.2	Limites des tests automatisés et tests manuels	27
3.4.3	Perspectives d'amélioration	27
3.5	Erreurs restantes	28
3.6	Liste des documents fournis	28
4	Conclusions	28
4.1	Objectifs atteints / non-atteints	28
4.2	Points positifs / négatifs	29
4.3	Difficultés particulières.....	29
4.4	Suites possibles pour le projet.....	30
5	Annexes.....	30
5.1	Résumé du rapport du TPI	30
5.2	Sources – Bibliographie.....	31
5.3	Journal de travail	33
5.4	Manuel d'installation	39
5.5	Manuel d'utilisation	40
5.6	Archives du projet.....	41

1 Analyse préliminaire

1.1 Introduction

Dans le cadre de ma formation d'informaticien au CPNV, je réalise mon TPI. Mon projet consiste à développer une application web nommée "MonParcours".

L'objectif principal de cette application est d'aider les utilisateurs à organiser leurs randonnées ou leurs voyages. Grâce à une interface interactive, l'utilisateur peut ajouter des étapes sur une carte, connecter ces points pour créer un itinéraire et personnaliser chaque étape avec des couleurs, des commentaires et des horaires spécifiques.

Pour réaliser ce projet, j'utilise les technologies modernes du web comme Vue.js pour le développement du site et la bibliothèque Leaflet pour la gestion de la carte. Toutes les données sont enregistrées localement sur le navigateur de l'utilisateur pour garantir une utilisation simple et rapide.

Ce document présente la planification, l'analyse technique et les étapes de réalisation de mon projet.

1.2 Glossaire

API (Interface de Programmation d'Application) : Ensemble de protocoles permettant à deux logiciels de communiquer.

Asynchrone (Programmation) : Méthode permettant d'exécuter des opérations (comme un appel API) en arrière-plan sans bloquer l'exécution du reste du code ou l'interface utilisateur.

Composants (Vue.js) : Éléments d'interface réutilisables et autonomes qui structurent l'application Vue 3.

DOM (Document Object Model) : Interface de programmation qui représente la structure d'un document HTML. Vue 3 manipule le DOM de manière réactive pour mettre à jour l'affichage.

Déploiement : Processus consistant à transférer l'application finale sur un serveur distant pour la rendre accessible en ligne aux utilisateurs.

ES6+ (ECMAScript 6) : Version moderne de JavaScript apportant des fonctionnalités avancées comme les classes, les promesses et les fonctions fléchées.

Framework : Structure logicielle offrant un ensemble d'outils et de bibliothèques pour faciliter le développement d'applications. Vue 3 est le framework principal utilisé ici.

JSON : Format utilisé pour stocker et échanger des données textuelles structurées.

Leaflet : Bibliothèque principale pour l'affichage et la manipulation de cartes interactives.

LocalStorage : Système de stockage local dans le navigateur pour sauvegarder les données du projet de manière persistante.

OpenStreetMap : Service fournissant les données cartographiques libres utilisées dans l'application.

Polyline : Ligne géométrique reliant plusieurs points de coordonnées sur la carte.

Réactivité : Capacité d'un système à mettre à jour automatiquement l'interface utilisateur dès que les données sources (état) sont modifiées.

UX (Expérience Utilisateur) : Qualité de l'interaction entre l'utilisateur et l'application, incluant la fluidité, l'esthétique et la facilité d'utilisation.

1.3 Objectifs

1.3.1 Objectifs techniques

Intégration de Leaflet et OpenStreetMap : Afficher une carte interactive fluide et gérer l'affichage de fonds de carte dynamiques via le protocole TileLayer. Il s'agit d'une solution Open Source ne nécessitant pas de clé API ni de gestion de quotas payants pour ce projet.

Utilisation de Vue 3 : Mettre en place une architecture réactive (Composition API) pour gérer l'état de l'application (liste des points, visibilité des lignes) et assurer une mise à jour instantanée du DOM.

Persistence des données : Implémenter l'utilisation du LocalStorage pour permettre la sauvegarde et la récupération des itinéraires sans avoir besoin d'une base de données externe.

Gestion des erreurs : Assurer la robustesse de l'application en gérant les cas limites, comme l'absence de données de géolocalisation ou la saisie de formats d'horaires invalides.

1.3.2 Objectifs fonctionnels

Interaction cartographique : Permettre à l'utilisateur de placer des marqueurs précis par un simple clic et de les supprimer facilement.

Personnalisation des étapes : Offrir la possibilité de modifier la couleur de chaque marqueur, d'ajouter un commentaire textuel et de définir une heure de passage.

Visualisation de l'itinéraire : Tracer automatiquement des lignes (Polylines) entre les points pour visualiser le trajet global du parcours.

Interface Intuitive : Proposer une barre latérale claire qui liste toutes les étapes de manière organisée et chronologique.

1.3.3 Objectifs pédagogiques

Approfondissement de JavaScript (ES6+) : Maîtriser la manipulation des tableaux, la programmation asynchrone et la gestion des objets JSON.

Gestion de projet via GitHub Projects : Mettre en pratique la méthodologie Agile en gérant le cycle de vie du projet à travers des sprints et des issues structurées.

Design et UX : Développer des compétences en conception d'interface utilisateur (UI) en créant des maquettes cohérentes et une navigation fluide.

Autonomie technique : Être capable de lire et d'appliquer une documentation technique complexe (comme celle de Leaflet.js)

1.4 Planification initiale

Candidat :	Ahmet KARABULUT
Titre du projet :	Planificateur de parcours pour excursion
Période :	Du 23.04.2026 au 20.05.2026
Temps imparti :	90 heures

Ce document présente la planification initiale de mon travail personnel de fin d'apprentissage (TPI). Le but de mon projet est de développer une application web frontend avec Vue.js pour permettre à un utilisateur de créer et de gérer des itinéraires sur une carte. J'ai organisé mon temps pour respecter les phases définies dans le cahier des charges.

Pour le suivi des tâches, l'outil GitHub Projects est utilisé via un tableau de bord (Kanban) permettant de visualiser en temps réel l'état d'avancement des activités. Afin d'assurer une livraison fluide et continue, j'ai adopté une stratégie de Trunk-based Development pour la gestion du code source. Cette méthode consiste à centraliser le développement sur une branche principale unique, favorisant ainsi une intégration rapide des fonctionnalités et une réduction des conflits techniques.

L'utilisation combinée de ces outils garantit une synchronisation parfaite entre la planification initiale et le développement technique de Mon Parcours. Cette organisation structurée permet d'anticiper les risques et de respecter les délais imposés par le cahier des charges.

1.4.1 Tableau récapitulatif

Phase de projet	Heures prévues
Analyse	16 h
Réalisation (Implémentation)	40 h
Tests	8 h
Documentation	16 h
Réserve pour imprévus (Problèmes techniques)	10 h
Total	90 h

1.4.2 Détail des activités

1.4.2.1 Phase d'Analyse (16h) (27.04.2026-29.04.2026)

- Analyse fonctionnelle (8h) : Analyse du cahier des charges, définition des besoins de l'utilisateur et des cas d'utilisation (Use Cases).
- Maquettes / Wireframes (3h) : Conception visuelle de l'interface utilisateur pour prévoir le design de l'application.
- Analyse technique (5h) : Choix des librairies (Leaflet pour la carte) et étude du stockage local (LocalStorage).

1.4.2.2 Phase de Réalisation (40h) (29.04.2026-18.05.2026)

- Installation du projet (3h) : Configuration de l'environnement de développement avec Vue.js et VSCode.
- Intégration de la carte (9h) : Affichage de la carte OpenStreetMap et gestion de la localisation de l'utilisateur.
- Gestion des marqueurs (11h) : Ajout de points sur la carte, modification des couleurs et ajout de commentaires par point.
- Outil de liaison (10h) : Développement de la logique pour relier les points entre eux et définir l'ordre de visite.
- Stockage local (7h) : Implémentation de la sauvegarde pour que l'utilisateur puisse retrouver son tracé plus tard.

1.4.2.3 Phase de Tests (8h) (27.04.2026-18.05.2026)

- Tests de fonctionnement (4h) : Vérification de toutes les fonctionnalités demandées dans le cahier des charges.
- Corrections et optimisations (4h) : Correction des erreurs (bugs) trouvées lors des tests.

1.4.2.4 Phase de Documentation (16h) (23.04.2026-20.05.2026)

- Journal de travail (6h) : Rédaction journalière de l'avancement du projet et des décisions prises.
- Rapport de projet (8h) : Rédaction du document final avec les explications techniques et les conclusions.
- Finalisation (2h) : Contrôle final des documents et création de l'archive (Code source + PDF).

1.4.2.5 Marge de Sécurité (10h)

Ces heures seront utilisées uniquement en cas de difficultés techniques majeures ou pour approfondir une fonctionnalité si le temps le permet.

1.4.3 Détail des Sprints

Afin d'assurer un suivi rigoureux et une gestion transparente via GitHub Projects, la phase de réalisation est structurée en deux sprints distincts de 20 heures chacun.

Cette approche itérative permet de valider les fonctionnalités par étapes :

1.4.3.1 *Sprint 1 : Mise en place de l'infrastructure et fonctions de base (MVP)*

- **Objectif** : Créer une application fonctionnelle capable d'afficher une carte et d'interagir avec l'utilisateur.

Tâches principales :

- Initialisation du projet avec Vue.js et intégration de la bibliothèque Leaflet.
 - Configuration de la carte OpenStreetMap (centrage par défaut et zoom).
 - Implémentation de la géolocalisation pour situer l'utilisateur en temps réel.
 - Développement de la fonction d'ajout de marqueurs (markers) par simple clic sur la carte.
 - Affichage d'une liste dynamique des points créés dans une barre latérale.
- **Livrable à la fin du sprint** : Une application capable de localiser l'utilisateur placer des points sur une carte.

1.4.3.2 *Sprint 2 : Logique métier, liaisons et persistance*

- **Objectif** : Enrichir l'expérience utilisateur et assurer la sauvegarde durable des données.
- **Tâches principales** :
 - Développement du système de liaison des points (tracé d'itinéraires/polylines).
 - Gestion de la personnalisation : choix des couleurs et ajout de commentaires pour chaque étape.
 - Implémentation de la logique de saisie horaire (format 24h) pour chaque point du parcours.
 - Mise en place de la sauvegarde automatique via le LocalStorage (persistance des données au rafraîchissement de la page).
 - Optimisation de l'interface utilisateur (UX/UI) et gestion des erreurs de saisie.
- **Livrable à la fin du sprint** : Une application complète permettant de planifier, personnaliser et sauvegarder un parcours d'excursion complet.

Voici à quoi ressemble mes kanbans pour chaque Sprint :

The screenshot shows a Jira Kanban board for the 'Trip Planner' project. The board is organized into three columns: Backlog, Ready, and In progress. Each column has a header with a status icon, a count, and an 'Estimate' field. Below the headers, items are listed in cards, each with a title, a description, and a 'TPI_Trip_Planner' label.

Column	Status	Count	Estimate	Item ID	Description
Backlog	Green circle	5 / 5	0	TPI_Trip_Planner #10	Draw lines (Polylines) between markers to show route
Backlog	Green circle	5 / 5	0	TPI_Trip_Planner #13	Implement time input (24h format) for each stop
Ready	Blue circle	2	0	TPI_Trip_Planner #11	Add a color picker to change marker color
Ready	Blue circle	2	0	TPI_Trip_Planner #12	Add a text field for comments on each marker
In progress	Yellow circle	1 / 3	0	TPI_Trip_Planner #9	Create a sidebar to list added markers

2 Analyse / Conception

2.1 Technologies utilisées

Pour répondre aux exigences techniques et assurer une interactivité ludique, les technologies suivantes ont été sélectionnées :

- Vue 3 : Framework principal utilisé pour la création d'une interface réactive et la gestion efficace des composants.
- JavaScript (ES6+) : Langage de base pour la logique asynchrone et les appels API.
- HTML5 / CSS3 : Pour la structure sémantique et la mise en forme stylisée de l'application.

2.2 Outils d'aide

Le développement est soutenu par une suite d'outils modernes, chacun jouant un rôle spécifique dans la réussite et l'organisation du projet :

VS Code (Visual Studio Code) : C'est l'éditeur de texte (IDE) principal utilisé pour l'écriture du code. Il est choisi pour sa légèreté et ses extensions qui facilitent le développement avec Vue 3, permettant de détecter les erreurs en temps réel.

Git / GitHub : Ces outils servent à la gestion de version. Git permet de sauvegarder l'historique des modifications, tandis que GitHub héberge le code en ligne. Cela sécurise le travail contre toute perte de données accidentelle.

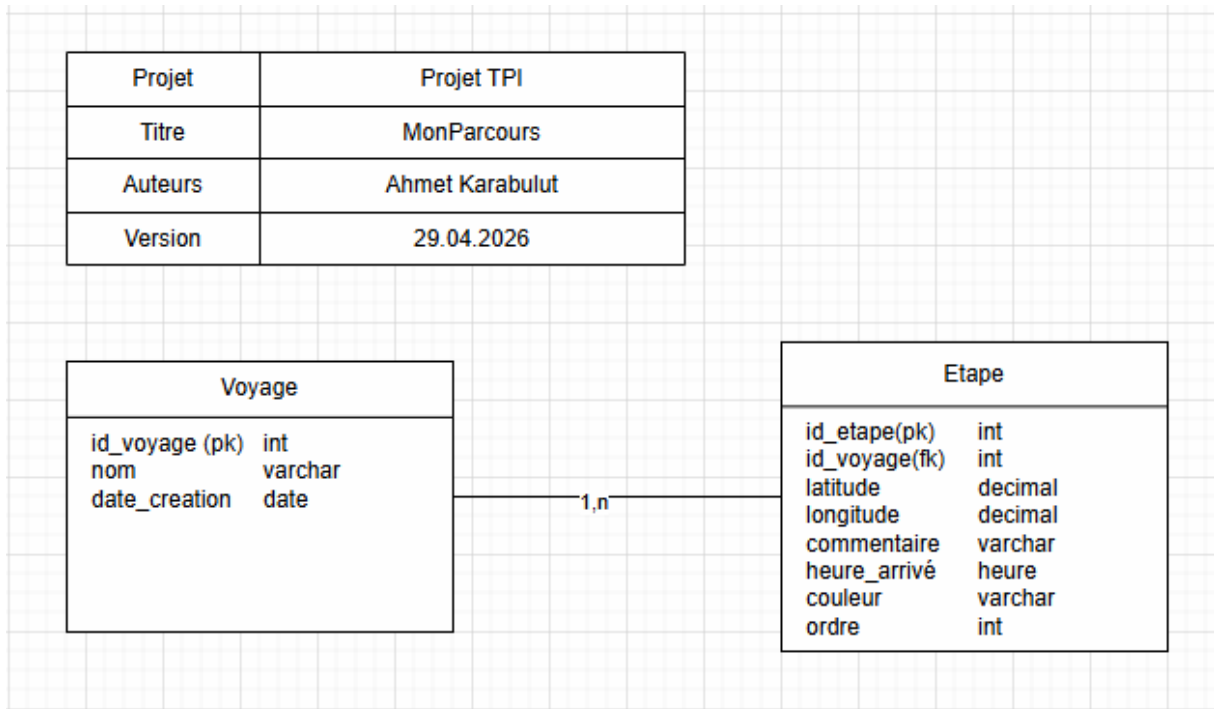
GitHub Projects : Utilisé pour la gestion de projet. Cet outil permet de créer un tableau de bord (style Kanban) pour suivre l'avancement des tâches (À faire, En cours, Terminé). Il aide à respecter les délais de chaque Sprint.

Vite : C'est l'outil de construction (build tool) qui accompagne Vue 3. Il permet de lancer le projet très rapidement durant le développement et d'optimiser les fichiers finaux pour qu'ils soient les plus légers possible pour l'utilisateur.

Figma / Balsamiq : Outils dédiés à la conception des maquettes. Balsamiq permet de dessiner rapidement des plans simples (wireframes) pour structurer l'interface, tandis que Figma aide à définir le design visuel précis et l'expérience utilisateur (UX).

Vercel / Netlify : Plateformes choisies pour le déploiement. Elles permettent de mettre l'application en ligne automatiquement dès que le code est mis à jour sur GitHub, rendant le projet accessible via un lien URL.

2.3 Modèle Conceptuel de Données (MCD)



Ce modèle représente la structure logique de l'application. L'objectif est de définir l'organisation des données avant le développement de l'application.

Description des entités :

- **Voyage** : Cette entité correspond à un parcours global. Elle contient un identifiant unique, un nom ainsi qu'une date de création.
- **Étape** : Cette entité représente un point précis affiché sur la carte. Chaque étape contient plusieurs informations nécessaires au fonctionnement de l'application :
 - Latitude : Coordonnée GPS du point.
 - Longitude : Coordonnée GPS du point.
 - Commentaire : Champ de texte libre permettant d'ajouter des informations personnalisées.
 - Heure d'arrivée : Heure prévue pour l'étape (format 24h).
 - Couleur : Couleur utilisée pour personnaliser le marqueur affiché sur la carte.
 - Ordre : Numéro permettant d'indiquer la position de l'étape dans le parcours.

Logique des relations :

Un voyage peut contenir plusieurs étapes (1,n), tandis qu'une étape appartient à un seul voyage (1,1). Cette relation permet d'organiser correctement les données du parcours.

Utilisation des champs spécifiques :

Le champ « ordre » est utilisé pour déterminer la séquence des étapes et permettre le tracé correct de l'itinéraire sur la carte. Les champs « commentaire », « heure d'arrivée »

et « couleur » permettent une personnalisation des différentes étapes selon les besoins de l'utilisateur.

Le modèle de données a été conçu de manière évolutive afin de permettre la gestion future de plusieurs voyages, même si l'implémentation actuelle utilise une structure simplifiée stockée dans le LocalStorage du navigateur.

2.4 Modèle Logique de Données (MLD)

Le choix du stockage des données repose directement sur les contraintes du cahier des charges. Celui-ci précise que l'utilisateur doit pouvoir stocker les données localement dans son navigateur afin de les retrouver lors d'une prochaine utilisation. Pour cette raison, le LocalStorage du navigateur a été utilisé à la place d'une base de données SQL externe.

Cette solution permet une sauvegarde simple des données sans nécessiter de serveur ou de backend.

Organisation des données (format JSON) :

Chaque étape du parcours est définie par un objet JavaScript structuré de manière précise. Voici le format utilisé dans mon code pour chaque nouvelle étape :

```
myMap.value.on('click',(e)=>{
  const newEtape={
    id:Date.now(), // milliseconde unique Integer
    lat: e.latlng.lat, //float
    lng:e.latlng.lng, //float
    name: "Point "+ (etapes.value.length+1), // string
    comment: "" ,// for commentaire // string
    color:'#4595fc', // string (hex)
    arrivalTime:'', // string
    order:etapes.value.length+1 // number(integer)
  };
  etapes.value.push(newEtape);
});
```

- ETAPE (id, lat, lng, name, comment, color, arrivalTime, order)

Propriété	Type de donnée	Justification technique
id	Number (Integer)	Utilisation de Date.now() qui génère le nombre de millisecondes écoulées. Cela garantit un identifiant unique à chaque clic de l'utilisateur.
lat	Number (Float)	Stocke la latitude sous forme de nombre décimal pour un positionnement précis sur la carte.
lng	Number (Float)	Stocke la longitude sous forme de nombre décimal.

name	String	Une chaîne de caractères permettant d'identifier l'étape (ex: "Point 1").
comment	String	Type texte pour permettre à l'utilisateur de saisir des notes libres sur son trajet.
color	String (Hex)	Stocke le code couleur au format hexadécimal (ex: #4595fc) pour personnaliser le marqueur.
arrivalTime	String	Stocke l'heure sous format texte (ex: "14:30") pour l'affichage dans la Sidebar.
order	Number (Integer)	Un nombre entier qui définit la position de l'étape dans la liste pour le tracé de l'itinéraire.

Chaque étape contient les informations nécessaires à l'affichage sur la carte ainsi qu'à la personnalisation des marqueurs.

Les données sont converties en JSON grâce à `JSON.stringify()` lors de la sauvegarde et récupérées avec `JSON.parse()` lors du chargement depuis le `LocalStorage`.

```
▼ {id: 1779193788298, lat: 46.785133585584425, lng: 6.608104705810548, name: 'Point 1', comment: '', ...}
  arrivalTime: ""
  color: "#4595fc"
  comment: ""
  id: 1779193788298
  lat: 46.785133585584425
  lng: 6.608104705810548
  name: "Point 1"
  order: 1
  ► [[Prototype]]: Object
```

2.5 Maquettes

Conformément à la planification du Sprint 1, j'ai réalisé une phase de prototypage à l'aide de l'outil Figma et Figma IA. Cette étape de conception graphique est essentielle pour définir l'ergonomie de l'application avant d'entamer le développement technique avec Vue 3. L'objectif est de créer une interface intuitive qui facilite la gestion de la carte et la visualisation de l'itinéraire.

1 PAGE D'ACCUEIL

La carte s'affiche directement à la localisation de l'utilisateur.

 Mon Excursion

 Sauvegarder

 Charger

 Effacer

 Ma position




© OpenStreetMap contributeurs

ÉTAPES (ordre de visite)


Aucun point ajouté.

 Ajouter un point

2 AJOUTER DES POINTS (ÉTAPES)

L'utilisateur ajoute plusieurs points sur la carte.

 Mon Excursion

 Sauvegarder

 Charger

 Effacer

 Ma position




© OpenStreetMap contributeurs

ÉTAPES (ordre de visite)

 1

Point de départ

Départ du parc

 09:00



 Ajouter un point

3 AJOUTER DES COMMENTAIRES, COULEURS ET HEURES

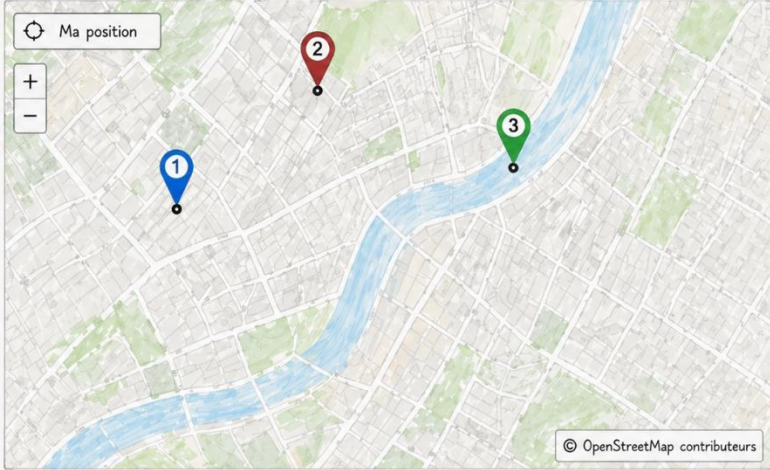
Pour chaque point, l'utilisateur ajoute un commentaire, une couleur et une heure d'arrivée.

Mon Excursion

Sauvegarder Charger Effacer

Ma position

+ -



© OpenStreetMap contributeurs

ÉTAPES (ordre de visite)

1 Point de départ

Départ du parc 09:00

2 Musée

Visite du musée 10:00

3 Café

Pause café 12:00

+ Ajouter un point

4 RELIER LES POINTS

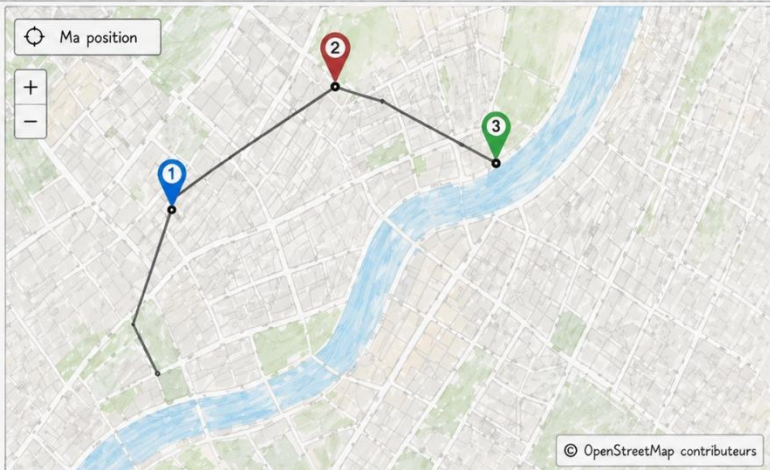
L'utilisateur relie les points entre eux pour déterminer l'ordre de visite.

Mon Excursion

Sauvegarder Charger Effacer

Ma position

+ -



© OpenStreetMap contributeurs

ÉTAPES (ordre de visite)

1 Point de départ

Départ du parc 09:00

2 Musée

Visite du musée 10:00

3 Café

Pause café 12:00

+ Ajouter un point

5 AFFICHAGE EN LISTE

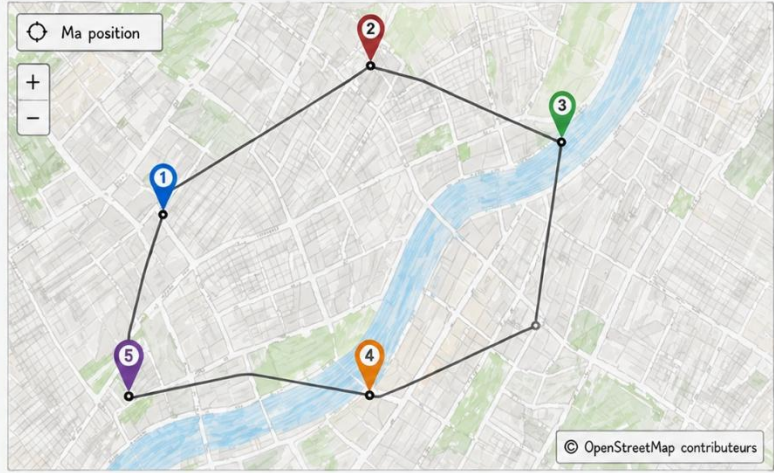
Les points sont affichés sous forme de liste dans l'ordre de visite, avec commentaires, couleurs et heures.

Mon Excursion

Sauvegarder Charger Effacer

Ma position

+ -



© OpenStreetMap contributeurs

ÉTAPES (ordre de visite)

1	Point de départ	Départ du parc	09:00	
2	Musée	Visite du musée	10:00	
3	Café	Pause café	12:00	
4	Belvédère	Superbe vue	14:00	
5	Parc	Fin de la visite	16:00	

+ Ajouter un point

6 SAUVEGARDE LOCALE

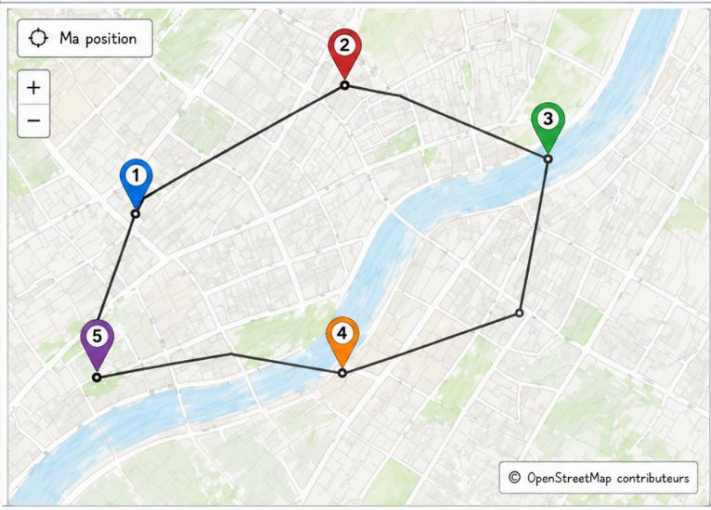
Une fois le tracé terminé, l'utilisateur peut sauvegarder son excursion dans le navigateur pour la retrouver plus tard.

Mon Excursion

Sauvegarder Charger Effacer

Ma position

+ -



© OpenStreetMap contributeurs

ÉTAPES (ordre de visite)

1	Point de départ	Départ du parc	09:00	
2	Musée	Visite du musée	10:00	
3	Café	Pause café	12:00	
4	Belvédère	Superbe vue	14:00	
5	Parc	Fin de la visite	16:00	

+ Ajouter un point

2.6 Stratégie de test

Pour garantir que mon application fonctionne correctement, j'ai mis en place une stratégie de vérification continue. Au lieu de tester tout à la fin, je préfère tester chaque fonctionnalité importante juste après l'avoir codée.

Ma démarche est la suivante :

1. **Développement** : J'ai d'abord réalisé l'intégralité des fonctionnalités prévues dans mon cahier des charges (carte, sidebar, stockage).
2. **Tests Manuels et visuels** : Une fois le code terminé, j'ai effectué des tests intensifs dans le navigateur. Cette étape m'a permis de vérifier l'ergonomie et de m'assurer que l'interface répondait correctement aux interactions de l'utilisateur.
3. **Tests automatisés (Stabilisation)** : Après avoir validé le fonctionnement visuel, j'ai écrit des tests unitaires avec Vitest. L'objectif de cette étape était de sécuriser la logique métier et de garantir que mon code est robuste et prêt pour la livraison finale.

2.6.1 Choix de l'outillage : Pourquoi Vitest ?

J'ai choisi Vitest parce que c'est un outil très moderne et rapide qui s'intègre parfaitement avec Vue 3 et Vite. C'est plus simple à utiliser que d'autres outils plus anciens, et cela me permet de voir les résultats des tests directement dans mon terminal.

2.6.2 Configuration du projet

Modification de vite.config.js : Pour que les tests puissent simuler un navigateur sans en ouvrir un réellement, j'ai configuré un environnement jsdom. Voici la configuration ajoutée :

```
// https://vite.dev/config/
export default defineConfig({
  plugins: [vue()],
  test: {
    // it must be jsdom for vitest
    environment: 'jsdom',
    globals: true,
  }
})
```

Ahmet Karabulut, 1 hour ago • se

Automatisation via package.json : Pour simplifier le flux de travail, j'ai ajouté un script personnalisé. Cela permet de lancer toute la suite de tests avec une simple commande :

```
"name": "monparcours",
"private": true,
"version": "0.0.0",
"type": "module",
  > Debug
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "test": "vitest",
  "preview": "vite preview"
},
"dependencies": {
  "leaflet": "^1.9.4",
  "vue": "^3.5.32"
},
"devDependencies": {
  "@vitejs/plugin-vue": "^6.0.6",
  "@vue/test-utils": "^2.4.9",
  "jsdom": "^29.1.0",
  "vite": "^8.0.10",
  "vitest": "^4.1.5"
}
}
```

Commande : npm run test

```
C:\Users\pro8gl\Desktop\Tpi\tpi_MonParcours\TPI_Trip_Planner\monparcours (main -> origin)
λ npm run test

> monparcours@0.0.0 test
> vitest
```

Mode Watch : Vitest reste en écoute et relance les tests dès qu'une modification est détectée dans le code source, garantissant un cycle de développement fluide.

2.6.3 Structure et Co-location (__tests__)

Les tests ne sont pas isolés dans un dossier racine distant, mais placés dans un répertoire __tests__ à l'intérieur de chaque dossier de composant.

- Visibilité : Le préfixe __ (Dunder => "Double Under score") permet de distinguer visuellement les fichiers d'outillage des fichiers de production dans l'arborescence.
- Maintenabilité : En gardant le test à côté de son composant, je m'assure que toute modification du code est immédiatement suivie d'une mise à jour de son test de spécification.

2.7 Risques techniques

La gestion des risques pour ce projet suit un cycle structuré afin d'améliorer la stabilité et les chances de succès, tout en augmentant la confiance dans les solutions techniques choisies.

2.7.1 Identification

Voici les principaux risques techniques identifiés pour le projet MonParcours :

R1 : Perte de données

Description : Risque que les données saisies par l'utilisateur (itinéraires, marqueurs) soient effacées lors de la fermeture du navigateur ou du rafraîchissement de la page.

R2 : Incompatibilité de la bibliothèque

Description : Problèmes d'intégration entre le framework Vue 3 et la bibliothèque cartographique Leaflet (conflits de manipulation du DOM).

R3 : Mauvaise saisie utilisateur

Description : Risque d'erreurs ou de bugs visuels si l'utilisateur saisit un format d'heure invalide (ex: 25h70) ou laisse des champs obligatoires vides.

R4 : Problème de géolocalisation

Description : Difficulté d'affichage si l'utilisateur refuse l'accès à sa position géographique ou si le navigateur ne supporte pas l'API de géolocalisation.

2.7.2 Analyse (Matrice des risques)

Risque	Probabilité (1-3)	Impact (1-3)	Score (P x I)
R1 (Perte données)	3	3	9
R2 (Enth. Leaflet/Vue)	1	2	2
R3 (Erreur Saisie)	3	2	6
R4 (Géo-accès)	2	1	2

2.7.3 Évaluation et Priorisation

Priorité Haute (Score 9) : La persistance des données (R1). C'est le risque critique car l'utilisateur perdrait tout son travail.

Priorité Moyenne (Score 6) : La validation des formulaires (R3). Important pour éviter les bugs visuels sur la carte.

Priorité Basse (Score 2) : L'intégration technique (R2) et l'accès à la géolocalisation (R4).

2.7.4 Traitement (Stratégies de réduction)

Pour R1 (LocalStorage) : Sauvegarde manuelle (bouton save) à chaque modification d'un point.

Pour R3 (Validation) : Utiliser des masques de saisie (input mask) pour le format 24h et des menus pour les couleurs.

Pour R4 (Géolocalisation) : Prévoir une position par défaut (ex: centre de la ville) si l'utilisateur refuse l'accès.

2.7.5 Suivi et contrôle

Chaque risque sera surveillé durant les phases de Sprint 1 et Sprint 2. Des tests unitaires simples seront effectués pour vérifier que le LocalStorage fonctionne correctement après chaque ajout de fonctionnalité.

2.8 Planification

Le tableau ci-dessous compare la répartition des heures initialement prévue lors du lancement du projet avec la réalité des heures investies et consignées rigoureusement dans le journal de travail :

Phase de projet	Heures prévues	Heures réelles	Écart / Justification
Analyse	16 h	17 h	+1h : Analyse approfondie des risques et conception du modèle MLD pour remplacer la base de données SQL standard.
Réalisation (Implémentation)	40 h	31.5 h	-8.5h : Efficacité accrue grâce à l'utilisation d'outils IA (Gemini) pour résoudre rapidement les bugs d'initialisation et de liaison de données (Props).
Tests	8 h	12.5 h	+4.5h : Effort intensifié sur l'automatisation des tests unitaires avec Vitest pour les composants MapContainer, EtapeItem et Sidebar.
Documentation	16 h	19 h	+3h : Rédaction exhaustive incluant le manuel d'utilisation illustré, le README professionnel et l'intégration des retours des experts.
Réserve pour imprévus	10 h	10 h	Répartie entièrement dans la gestion des difficultés techniques complexes (précision géolocalisation Desktop/IP et conflits d'affichage).
Total	90 h	90 h	Respect strict de l'enveloppe globale de 90 heures allouée pour le TPI.

Analyse descriptive de la planification

Comme l'atteste le journal de travail, le projet « Mon Parcours » a été complété en respectant scrupuleusement la limite réglementaire des 90 heures. Toutefois, la répartition interne des efforts a légèrement différé de la planification initiale pour des raisons techniques constructives :

1. Optimisation du temps de développement (Réalisation) : La phase de codage pur a nécessité 31.5 heures au lieu des 40 heures initialement budgétisées. Ce gain de temps substantiel s'explique par l'intégration méthodique des technologies modernes et l'appui ciblé de l'intelligence artificielle (Gemini/ChatGPT). Ces outils ont permis de diagnostiquer instantanément des erreurs bloquantes de

compilation, telles que le cycle de vie de Leaflet dans `onMounted` ou les problèmes de sensibilité à la casse (Case-Sensitivity) dans Vue 3.

2. Réinvestissement dans la qualité (Tests et Documentation) : Les heures économisées lors du développement ont été intelligemment réallouées afin de maximiser la robustesse de l'application. La phase de tests a ainsi bénéficié de 4.5 heures supplémentaires, dédiées à la mise en place d'une couverture complète de tests unitaires avec Vitest pour l'ensemble des composants stratégiques (`MapContainer.vue`, `RouteStorage.vue`, `Sidebar.vue`, `EtapItem.vue`). De même, la documentation technique a été enrichie pour garantir un guide d'installation et d'utilisation transparent pour les utilisateurs.
3. Utilisation de la réserve pour imprévus : La marge de sécurité de 10 heures a été entièrement mobilisée pour restructurer la logique applicative face aux contraintes du monde réel. Notamment, la gestion de la faible précision GPS sur les navigateurs Desktop (filtrage conditionnel de la marge d'erreur si supérieure à 150m) et le déploiement automatisé sur la plateforme Vercel ont été intégrés durant ces phases charnières.

En conclusion, bien que des ajustements dynamiques aient été nécessaires entre les phases, la méthodologie Agile et le suivi par sprints ont permis de livrer un produit final totalement conforme au cahier des charges, dans les délais impartis.

2.9 Dossier de conception

2.9.1 Architecture modulaire des composants visuels

Mon application utilise une structure hiérarchique de composants pour assurer une grande flexibilité :

MapContainer.vue : Le composant parent qui affiche la carte OpenStreetMap.

Sidebar.vue : Le panneau latéral qui contient la liste des étapes.

EtapItem.vue : Un composant spécialisé qui gère l'affichage et l'édition d'une seule étape (heure, couleur, commentaire). Cette modularité permet de gérer chaque point du trajet de manière isolée et précise.

RouteStorage.vue : Le module responsable de la communication avec le `LocalStorage` pour la sauvegarde.

2.9.2 Routage (Logique de l'itinéraire)

Dans ce projet, le terme 'Routage' a deux significations importantes. Techniquement, j'ai utilisé une approche de Single Page Application (SPA) : au lieu de recharger des pages HTML, l'application met à jour dynamiquement les composants Vue.

Sur la carte, le routage est géré par la logique du bouton 'Draw Lines'. Ce bouton parcourt le tableau des étapes, récupère chaque coordonnée (latitude et longitude) et utilise une fonction L.polyline de Leaflet pour relier les points. Cela permet de visualiser l'itinéraire complet de manière fluide et instantanée.

2.9.3 Gestion des appels de données (OpenStreetMap & Geolocation)

Pour l'affichage des cartes, j'utilise les serveurs de OpenStreetMap. La communication se fait via la bibliothèque Leaflet qui récupère les images de la carte (tiles) en temps réel. Pour la position de l'utilisateur, j'utilise l'API Geolocation du navigateur. Cela permet de récupérer les coordonnées GPS de l'appareil pour centrer la carte et placer un marqueur 'You are here' avec une précision dynamique.

2.9.4 Gestion de l'Interface Utilisateur (UI/UX)

L'interface est pensée pour être simple. À droite, la Sidebar contient les contrôles et les informations textuelles (heure, commentaires, couleurs). À gauche, la carte occupe la majorité de l'écran pour une meilleure visibilité. J'ai ajouté des retours visuels, comme des messages d'alerte lors de la sauvegarde ou du chargement des données, pour que l'utilisateur sache toujours si son action a réussi.

3 Réalisation

Produit : Mon Parcours

Version: 1.0.0 (Version initial)

3.1 Dossier de réalisation

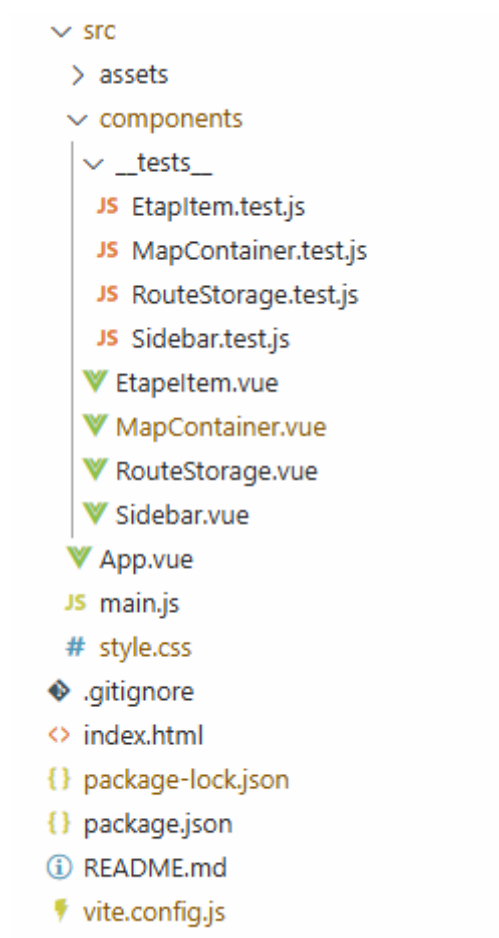
Environnement technique (Matériel et Logiciels) :

- Matériel : Dell PC du CPNV
- Système d'exploitation : Windows 11 Éducation
- Outils de développement : Node.js (v22.18.0), NPM (11.6.2), VS Code.

Librairies externes :

- Vue.js (3.5.32) : Framework JavaScript progressif utilisé pour construire l'interface utilisateur réactive.
- Leaflet (1.9.4) : Bibliothèque JavaScript open-source utilisée pour l'affichage de la carte interactive et la manipulation des marqueurs géographiques.
- Vite (7.2.4) : Outil de build ultra-rapide servant de serveur de développement et permettant la compilation du projet.
- Vitest (6.0.1) & Vue Test Utils (2.4.6) : Framework de test et utilitaires utilisés pour garantir la fiabilité de la logique des composants via des tests unitaires.

3.1.1 Structure du repository



3.2 Construction de la documentation

La documentation de ce projet a été conçue pour permettre à n'importe quel utilisateur ou développeur de comprendre, d'installer et d'utiliser l'application rapidement. Elle se divise en trois parties complémentaires :

- Documentation technique : J'ai intégré des commentaires directement dans le code source pour expliquer les fonctions complexes, notamment pour la gestion du LocalStorage et les calculs de l'itinéraire avec Leaflet. Cela facilite la maintenance future du projet.
- Manuel d'installation : J'ai rédigé un guide étape par étape (Annexe 5.4) pour configurer l'environnement de développement. Ce document explique comment cloner le projet et installer les dépendances avec NPM pour rendre l'application opérationnelle en quelques minutes.
- Manuel d'utilisation : Pour accompagner l'utilisateur final, (Annexe 5.5). Ce manuel explique de manière simple comment ajouter des étapes, personnaliser les trajets et sauvegarder son parcours. L'objectif était de rendre l'application accessible même pour une personne non technique.

3.3 Résultat des tests effectués

3.3.1 Tests unitaires :

```
C:\Users\pr08glt\Desktop\Tpi\tpi_MonParcours\TPI_Trip_Planner\monparcours (main -> origin)
λ npm run test

> monparcours@0.0.0 test
DEV v4.1.5 C:/Users/pr08glt/Desktop/Tpi/tpi_MonParcours/TPI_Trip_Planner/monparcours

stdout | src/components/__tests__/MapContainer.test.js > MapContainer.vue - Detailed Unit Tests >
User refused location. Staying in Yverdon

✓ src/components/__tests__/Sidebar.test.js (7 tests) 56ms
✓ src/components/__tests__/EtapItem.test.js (10 tests) 116ms
✓ src/components/__tests__/MapContainer.test.js (15 tests) 139ms
✓ src/components/__tests__/RouteStorage.test.js (6 tests) 53ms

Test Files 4 passed (4)
Tests 38 passed (38)
Start at 11:55:41
Duration 3.21s (transform 413ms, setup 0ms, import 1.02s, tests 364ms, environment 4.68s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

3.3.2 Tests manuels :

No	Scénario de test	Résultat attendu	État
Géolocalisation et Permissions (Navigateur)			
1	Demande de permission : Cliquer sur le bouton "My Position".	Le navigateur affiche une notification demandant l'autorisation d'accéder à la position.	OK
2	Succès Géolocalisation : Accepter la permission (Allow).	La carte se déplace et se centre automatiquement sur ma position réelle.	OK
3	Refus Géolocalisation : Refuser la permission (Block).	Une alerte s'affiche et la carte reste centrée (ou revient) sur Yverdon-les-Bains.	OK
4	Géolocalisation : (Simulation GPS)	La carte se centre avec une précision < 50m lors d'une simulation technique.	OK
5	Géolocalisation réelle : (Desktop/IP)	Affichage de l'avertissement si la précision est faible (> 500m).	OK
Interaction avec la Carte et les Points			
6	Ajout d'une étape : Cliquer sur une zone vide de la carte.	Un marqueur apparaît exactement à l'endroit du clic.	OK
7	Détails d'un point : Cliquer sur un marqueur déjà existant sur la carte.	Une popup s'ouvre et affiche le nom du point (ex: "Point 1").	OK
Édition et Personnalisation			
8	Saisie de l'heure d'arrivée : Entrer une heure spécifique (ex: 14h30) pour une étape.	Le composant input[type="time"] accepte la valeur et la mémorise correctement.	OK
9	Ajout d'un commentaire : Écrire une note ou une adresse dans le champ "Commentaire".	Le texte est bien enregistré dans l'objet de l'étape et reste visible lors de la navigation.	OK
10	Changement de couleur : Cliquer sur l'une des 8 couleurs proposées dans la palette.	La couleur sélectionnée devient active (bordure accentuée) et l'objet étape est mis à jour.	OK

11	Synchronisation visuelle : Modifier la couleur d'une étape dans la Sidebar.	Le marqueur (circleMarker) correspondant sur la carte change de couleur en temps réel.	OK
Tracé de l'itinéraire			
12	Tracé de la route : Cliquer sur le bouton "Draw Lines" après avoir ajouté au moins 2 points.	Une ligne continue (polyline) est tracée sur la carte, reliant tous les points selon leur ordre de création (#1, #2, #3...).	OK
13	Ordre du tracé : Vérifier visuellement si la ligne suit bien la séquence numérique des étapes.	La ligne part du point #1 vers le point #2, puis vers le point #3, sans croisement incohérent.	OK
14	Cas d'erreur (Négatif) : Cliquer sur "Draw Lines" avec un seul point ou aucun point.	Une alerte s'affiche (ex : "Add at least 2 points") et aucune ligne n'est tracée sur la carte.	OK
15	Mise à jour dynamique du tracé : Ajouter un nouveau point (#4) et cliquer à nouveau sur le bouton "Draw Lines".	L'ancien tracé est automatiquement supprimé de la carte et un nouvel itinéraire incluant tous les points (1 à 4) est dessiné.	OK
Affichage de la Liste			
16	Visibilité de la liste : Ajouter plusieurs points sur la carte et observer la Sidebar.	Chaque point ajouté sur la carte apparaît instantanément sous forme de carte (Etapeltem) dans la liste latérale.	OK
17	Numérotation séquentielle : Vérifier l'ordre des points dans la liste.	Les points sont numérotés automatiquement (#1, #2, #3...) selon leur ordre d'ajout, facilitant la lecture de l'itinéraire.	OK
18	Synchronisation totale : Modifier une information (couleur, heure) dans la liste.	La modification est visible dans la liste et se répercute sur les données globales de l'itinéraire.	OK
19	Scroll de la liste : Ajouter un grand nombre de points (ex: 10 points).	Une barre de défilement (scroll) apparaît dans la Sidebar, permettant de naviguer dans la liste sans déformer l'interface.	OK
Persistance des Données			
20	Sauvegarde de l'itinéraire : Ajouter des points avec commentaires et couleurs, puis cliquer sur "Save".	Une alerte de confirmation "Saved!" apparaît. Les données sont stockées techniquement dans le localStorage du navigateur.	OK
21	Persistance après rafraîchissement : Actualiser la page (F5) ou fermer/rouvrir le navigateur.	L'application redémarre à zéro (état initial vide), ce qui est normal avant le chargement.	OK
22	Chargement des données : Cliquer sur le bouton "Load" après avoir rafraîchi la page.	Tous les points sauvegardés réapparaissent sur la carte et dans la Sidebar avec leurs couleurs, heures et commentaires d'origine.	OK
23	Gestion d'un stockage vide : Cliquer sur "Load" alors qu'aucune sauvegarde n'a été effectuée.	Une alerte "No route" s'affiche pour informer l'utilisateur qu'aucune donnée n'est disponible.	OK
24	Sauvegarde vide (Négatif) : Cliquer sur "Save" sans avoir ajouté de points.	Une alerte "Nothing to save! Please add some points first" apparaît. Rien n'est enregistré.	OK
25	Chargement vide (Négatif) : Supprimer manuellement les données (ou sur un nouveau navigateur) et cliquer sur "Load".	Une alerte "No route found in storage!" s'affiche de manière sécurisée.	OK

3.4 Couverture des tests

3.4.1 Méthodologie de test des composants

"Pour ce projet, j'ai choisi de tester chaque composant selon sa responsabilité.

Logique de données : Pour RouteStorage.vue, ma méthode consiste à simuler le comportement du navigateur. J'ai écrit des scripts qui vérifient si le programme réagit correctement quand le stockage est plein ou vide.

Communication inter-composants : J'ai testé si les informations (couleurs, coordonnées) passent bien du menu latéral (Sidebar) vers la carte (MapContainer) via les événements Vue (Emit).

Validation unitaire : Chaque petite fonction, comme le calcul du numéro de l'étape, a été testée avec Vitest pour éviter des erreurs de calcul simples."

3.4.2 Limites des tests automatisés et tests manuels

"L'utilisation de la bibliothèque Leaflet crée une limite importante pour les tests automatisés.

Le problème du rendu : Vitest peut vérifier si une donnée existe, mais il ne peut pas 'voir' si le marqueur bleu est bien positionné sur la ville d'Yverdon.

L'interaction humaine : Cliquer sur une carte pour ajouter un point est une action très visuelle. Seul un test manuel peut confirmer que l'interface est fluide et que les couleurs changent instantanément sans ralentissement.

Les capteurs : La simulation de la géolocalisation réelle dans un test automatique est complexe. Le test manuel est ici indispensable pour vérifier la demande de permission du navigateur."

3.4.3 Perspectives d'amélioration

"Si je devais continuer le développement, j'ajouterais les éléments suivants pour sécuriser le code :

Tests d'intégration globaux : Utiliser un outil pour tester le scénario complet : 'Ajouter 2 points -> Tracer la ligne -> Sauvegarder -> Recharger'. Actuellement, je teste ces étapes séparément.

Contrôle des types de données : Ajouter des vérifications pour empêcher l'utilisateur d'entrer des caractères spéciaux dans les commentaires, ce qui pourrait casser le format JSON.

Tests de performance : Tester la fluidité de la carte si l'utilisateur ajoute plus de 50 ou 100 points, pour voir si l'application reste rapide."

3.5 Erreurs restantes

À ce stade du développement, toutes les fonctionnalités principales (ajout de points, tracé de la route, stockage local) ont été testées et validées. Il n'y a aucune erreur critique (bloquante) connue.

3.6 Liste des documents fournis

La livraison finale de la « Mon Parcours » se compose des éléments suivants :

Le rapport de projet (v1.0) : Ce document présent dans sa version finale.

Le code source : L'intégralité du projet est hébergée sur le dépôt GitHub suivant : https://github.com/akrblt/TPI_Trip_Planner/blob/main/monparcours/README.md

Le manuel d'installation : Il se trouve dans l'Annexe 5.4 (et également résumé dans le fichier README.md du dépôt). Ce manuel détaille la procédure pour installer le projet à partir de zéro, incluant le clonage du dépôt, l'installation des dépendances via NPM.

Le manuel d'utilisation : Il se trouve dans l'Annexe 5.5. Ce guide illustré permettant de comprendre comment lancer le serveur de développement local (npm run dev) et comment interagir avec l'interface pour ajouter des étapes sur la carte, personnaliser les détails dans la Sidebar et tracer l'itinéraire final.

4 Conclusions

4.1 Objectifs atteints / non-atteints

Tous les objectifs définis dans le cahier des charges ont été pleinement réalisés :

Affichage de la carte : Intégration réussie d'une carte interactive fluide via Leaflet.

Géolocalisation : L'utilisateur peut localiser sa position actuelle en temps réel sur la carte.

Gestion des points : Possibilité d'ajouter des étapes par simple clic ou via une recherche.

Personnalisation complète : Pour chaque point, l'utilisateur peut définir une couleur, ajouter un commentaire et spécifier une heure d'arrivée.

Création d'itinéraires : Génération automatique d'une route (polyline) reliant tous les points ajoutés.

Persistance et Chargement : Les données sont sauvegardées dans le LocalStorage. Une fonctionnalité de chargement (Load) permet de restaurer l'itinéraire complet après le rafraîchissement de la page.

4.2 Points positifs / négatifs

Points positifs :

Fiabilité avec Vitest : L'implémentation de tests unitaires avec Vitest et Vue Test Utils a été un atout majeur. Cela m'a permis de garantir la robustesse de la logique de calcul d'itinéraire et de détecter rapidement les erreurs lors de l'ajout de nouvelles fonctionnalités réactives.

Qualité documentaire : J'ai investi beaucoup de soin dans la rédaction de la documentation technique. Mon objectif était de rendre un travail final très qualitatif, clair et soigné, en dépassant mes standards habituels pour offrir une lecture fluide aux experts.

Finalisation complète : Un point très positif est d'avoir réussi à livrer 100% des fonctionnalités prévues dans le Cahier des Charges, notamment la gestion complexe de la persistance via LocalStorage et l'affichage dynamique des itinéraires sur la carte.

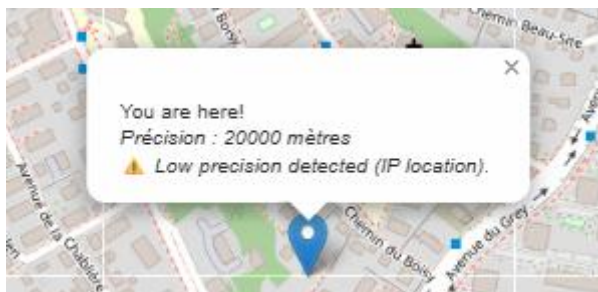
Points négatifs :

Précision des tracés (Routing) : Le tracé entre les points est une ligne droite (Polyline). Pour un projet de "Trip Planner", il aurait été plus réaliste de suivre les routes réelles via une API de routing (comme OSRM). Cependant, la mise en œuvre de cette API et la gestion des quotas de requêtes auraient ajouté une incertitude technique que j'ai préféré éviter pour garantir la stabilité du projet final.

4.3 Difficultés particulières

Gestion et filtrage de la précision GPS (Accuracy) : L'une des difficultés majeures a été le traitement des données de géolocalisation, particulièrement sur les navigateurs de bureau (Desktop). Ces derniers utilisent souvent l'adresse IP pour localiser l'utilisateur, ce qui génère une marge d'erreur très élevée (accuracy > 500m).

Le problème : Afficher systématiquement une précision de "2000 mètres" à l'utilisateur peut être déroutant et dégrader l'expérience utilisateur (UX) si le point sur la carte semble visuellement correct.



La solution technique : J'ai dû implémenter une logique de filtrage conditionnel. J'ai défini un seuil de 150 mètres : en dessous de cette valeur, l'information technique est masquée pour ne pas encombrer l'interface. Au-delà, l'information est affichée pour avertir l'utilisateur. De plus, j'ai ajouté une alerte spécifique lorsque la précision est jugée

"critique" (accuracy > 500m), permettant d'expliquer la source potentielle de l'erreur (localisation via IP vs GPS). Cela a nécessité une gestion rigoureuse des conditions de rendu dans Vue 3.

4.4 Suites possibles pour le projet

Moteur de recherche d'itinéraire : Intégrer une API (comme OSRM ou GraphHopper) pour que la route suive le tracé réel des routes et chemins.

Exportation de données : Permettre le téléchargement de l'itinéraire au format JSON ou CSV pour une utilisation externe.

Notifications : Ajouter un système d'alerte basé sur l'heure d'arrivée prévue pour chaque étape.

5 Annexes

5.1 Résumé du rapport du TPI

Dans le cadre de ma formation d'informaticien au CPNV, j'ai réalisé un Travail Pratique Individuel (TPI) nommé « MonParcours ».

Le but du projet est de développer une application web permettant aux utilisateurs de créer et organiser des itinéraires sur une carte interactive. L'utilisateur peut ajouter des étapes, personnaliser les points avec des couleurs, des commentaires et des heures, puis sauvegarder les données localement dans le navigateur.

Pour réaliser ce projet, j'ai utilisé plusieurs technologies modernes :

- Vue 3 pour l'interface utilisateur réactive
- Leaflet et OpenStreetMap pour la gestion de la carte
- LocalStorage pour la sauvegarde des données
- Vitest pour les tests unitaires

Le projet a été organisé avec une méthode Agile utilisant GitHub Projects et des sprints de développement. Cette organisation m'a permis de suivre l'avancement du projet et de respecter les délais.

Durant le développement, plusieurs difficultés techniques ont été rencontrées, notamment la gestion de la géolocalisation et la précision GPS. Des solutions ont été mises en place pour améliorer l'expérience utilisateur et garantir la stabilité de l'application.

Des tests manuels et automatisés ont été réalisés afin de vérifier le bon fonctionnement des fonctionnalités principales :

- Ajout des étapes,
- Tracé des itinéraires,

- Personnalisation des points,
- Sauvegarde et chargement des données.

Le projet final répond aux objectifs définis dans le cahier des charges. Cette expérience m'a permis d'améliorer mes compétences en développement frontend, en gestion de projet, en tests logiciels et en documentation technique.

5.2 Sources – Bibliographie

Pour la réalisation de ce projet, j'ai utilisé diverses ressources techniques et documentations officielles afin de garantir le respect des standards de développement actuels.

Documentations officielles :

Vitest : <https://vitest.dev/> – Documentation pour la mise en place des tests unitaires.

Vite.js : <https://vitejs.dev/> – Guide de configuration pour l'outil de build et le serveur de développement.

Vue.js 3 : <https://vuejs.org/> – Documentation principale pour la structure des composants et la réactivité.

Leaflet.js : <https://leafletjs.com/examples/quick-start/> – Guide d'intégration et exemples (Quick Start) pour la manipulation de la carte interactive et des marqueurs.

MDN (Mozilla Developer Network): <https://w3c.github.io/geolocation/#dom-navigator-geolocation> – Spécifications techniques pour l'implémentation du système de géolocalisation (navigator.geolocation).

OpenStreetMap Wiki : https://wiki.openstreetmap.org/wiki/Main_Page – Documentation de référence pour l'utilisation et les limites des tuiles cartographiques libres de droits.

Environnement de développement et Outils (Outils Techniques)

GitHub & Git : Plateforme d'hébergement du code source, utilisée pour le versionnage régulier du projet, la gestion des commits et le suivi des fonctionnalités (Issues).

Visual Studio Code : Éditeur de code principal

Communautés techniques et Intelligence Artificielle (Supports Numériques)

Stack Overflow : <https://stackoverflow.com/> – Plateforme communautaire consultée pour la résolution de bugs spécifiques liés à la réactivité JavaScript et aux ajustements CSS.

Intelligences Artificielles (Gemini & ChatGPT) : Utilisées comme assistants virtuels pour l'aide à la formulation textuelle du rapport, la relecture grammaticale et le support technique lors de la résolution de bugs algorithmiques.

Aides humaines (Ressources Humaines)

M. Hurni : Conseils et retours réguliers, réponses à mes questions.

5.3 Journal de travail

Date	Description de l'activité	Remarque / complément	Duré
23.04.2026	Rencontre avec l'Expert 1 et présentation du projet.	Entretien avec un expert au sujet du processus TPI et des critères d'évaluation. Lecture des objectifs techniques. Signature du document pour acceptation du cahier de charge.	1h
23.04.2026	Établissement de la planification initiale.	Répartition des 90 heures. Intégration d'une marge de sécurité de 10 heures pour les imprévus.	3h
27.04.2026	Analyse	Analyse approfondie du cahier des charges et étude de faisabilité technique (Vue 3, Leaflet).	2.5h
27.04.2026	Organisation du projet	Création du repository GitHub et configuration du tableau de bord sur GitHub Projects.	0.5h
27.04.2026	Planification technique	Définition des deux sprints de réalisation (20 heures chacun). Le Sprint 1 sera focalisé sur le carte et marqueurs et le Sprint 2 sur les fonctionnalités avancées (liaisons, LocalStorage).	2h
27.04.2026	Analyse	Identification et évaluation des risques techniques (matrice d'impact).	2h
28.04.2026	Prototypage UI/UX	Conception de la maquette principale sur Figma (Layout) : intégration de la carte interactive et de la barre latérale pour la gestion des étapes.	2h30
28.04.2026	Initialisation du projet	Fin de la phase d'analyse et début de la réalisation. Clonage du dépôt GitHub et initialisation de l'environnement de développement avec Vite et Vue 3. Installation de la bibliothèque cartographique Leaflet via NPM. Nettoyage des composants de base et structuration de l'arborescence du projet (dossiers composants et assets). Configuration globale du projet pour inclure les fichiers CSS nécessaires au rendu de la carte.	1h
28.04.2026	Développement du composant Map	Création du composant MapContainer.vue. Implémentation de la logique d'initialisation (L.map) centrée sur Yverdon-les-Bains et configuration de la couche de tuiles (Tile Layer) OpenStreetMap.	2h
28.04.2026	Optimisation de l'affichage	Résolution des problèmes de mise en page liés aux styles par défaut de Vite. Application de styles (MapContainer.vue) CSS (100vh, 100vw) pour	0.5h

		garantir un affichage de la carte en mode plein écran sans marges résiduelles.	
29.04.2026	Recherche technique	Recherche sur l'utilisation de la géolocalisation en JavaScript. Consultation de la documentation MDN (Mozilla Developer Network) pour comprendre navigator.geolocation.	2h
29.04.2026	Implémentation et erreur	Tentative d'affichage de la position sur la carte. Une erreur est apparue dans la console: Uncaught TypeError: Cannot read properties of undefined (reading 'setView').	1.5h
29.04.2026	Recherche et résolution	Utilisation de l'outil IA (Gemini) pour comprendre l'erreur. L'IA a expliqué que la variable map n'était pas accessible en dehors de la fonction onMounted. Correction avec l'utilisation de ref().	0.5h
29.04.2026	Analyse et conception de la structure des données	Une réflexion a été menée pour organiser les informations sans système de compte utilisateur. L'accent a été mis sur la relation entre un voyage et ses étapes. Les attributs nécessaires comme la couleur, le commentaire et l'heure d'arrivée ont été définis pour chaque point, conformément au cahier des charges.	2h
29.04.2026	Modèle Conceptuel de Données (MCD)	Un schéma conceptuel a été dessiné pour représenter les entités "Voyage" et "Étape". Ce travail permet de visualiser comment les objets JavaScript seront structurés avant de commencer la programmation du stockage local (localStorage).	1h
30.04.2026	Analyse du stockage local (LocalStorage)	Une réflexion a été menée pour remplacer une base de données SQL par un stockage local. La structure des objets JSON a été définie pour organiser les voyages et les étapes. L'objectif était de garantir une sauvegarde fiable des données sans serveur externe.	2h
30.04.2026	Modèle Logique de Données (MLD)	Le Modèle Logique de Données (MLD) a été rédigé pour le rapport. La description explique comment les identifiants lient les étapes aux voyages dans le format JSON.	1h
30.04.2026	Documentation		3h

04.05.2026	Analyse	Analyse de la fonctionnalité "Ajout de points" et planification de la structure modulaire.	1h
04.05.2026	Pause Tartine		0.5h
04.05.2026	Conception	Création des composants Sidebar.vue et EtapeItem.vue pour mieux organiser le code. L'idée est de séparer la carte et la liste des points.	2h
04.05.2026	Liaison des données (Props)	Mise en place de la communication entre les composants. Utilisation des Props pour transmettre les coordonnées et les informations des points depuis MapContainer vers la liste (Sidebar). L'objectif est de centraliser les données pour assurer une mise à jour dynamique de l'interface. Utilisation de l'outil IA (Gemini) pour liaison des données.	1h
04.05.2026	Résolution de bug technique	Analyse de l'erreur "Uncaught TypeError: Cannot read properties of null (reading 'on')". Le problème se trouvait dans la fonction d'initialisation de la carte. L'événement myMap.value.on('click') a été déplacé à l'intérieur de onMounted pour s'assurer que l'objet Leaflet est bien créé avant l'interaction.(MapContainer.vue) . Utilisation de l'outil IA (Gemini) pour comprendre l'erreur et trouver une solution.	1.5h
04.05.2026	Ajouter plusieurs points	Programmation de la fonction d'ajout de plusieurs points sur la carte. Chaque clic crée un marqueur et un nouvel élément dans la liste.	1.5h
05.05.2026	Développement de la fonction de commentaire	Implémentation d'un champ de texte dans l'interface pour permettre l'ajout de notes sur chaque étape du parcours. Validation de la saisie utilisateur et mise à jour de l'affichage dans la liste latérale.	1.5h
05.05.2026	Correction technique (Bug case-sensitivity)	Identification d'une erreur de syntaxe dans le fichier Sidebar.vue. L'élément Point était écrit avec une majuscule à certains endroits et avec une minuscule (point) à d'autres. Cette différence provoquait une erreur de compilation dans Vue.js.	0.5h
05.05.2026	Standardisation du code	Correction de l'élément en utilisant le minuscule point dans le fichier Sidebar.vue pour assurer la cohérence du projet. Vérification du fonctionnement de la liste après la correction.	0.5h

05.05.2026	Gestion des couleurs des marqueurs	Création d'un système de sélection de couleur (Bleu, Rouge, Vert) pour différencier les types de points sur la carte. Modification dynamique des icônes Leaflet lors du choix de l'utilisateur.	1h
05.05.2026	Analyse et correction d'un bug d'affichage	Identification d'un problème avec le changement de couleur des points. Malgré l'absence d'erreurs dans la console, les couleurs restaient identiques sur la carte. Après une recherche avec l'aide de ChatGPT pour comprendre la communication entre les composants Vue.js, une erreur de syntaxe a été trouvée.	1.5h
05.05.2026	Résolution technique	Remplacement de l'événement @color-changed par @change-color dans le code (Sidebar.vue). Cette modification a permis de lier correctement le sélecteur de couleur à la fonction de mise à jour des marqueurs.	0.5h
06.05.2026	Ajouter heure	Ajout d'un champ de saisie de type "time" dans l'interface. Modification de la structure des données pour enregistrer cette information.	2h
06.05.2026	Correction format (HH :MM)	Vérification du bon affichage du time dans la liste des étapes. Correction d'un bug mineur sur le format d'affichage. (HH :MM)	1h
06.05.2026	Recherche pour relier les points	Analyse des solutions pour relier les points sur la carte OpenStreetMap. Recherche sur l'utilisation des polygones pour tracer l'itinéraire entre les étapes.	1h
06.05.2026	Implémentation	Développement de la fonction de tracé automatique avec polygones entre les marqueurs. Mise à jour visuelle de la carte lors de l'ajout d'un nouveau point.	1h
06.05.2026	Implémentation	Ajout de la numérotation des points sur la carte afin d'indiquer l'ordre des étapes. Utilisation des tooltips de Leaflet pour afficher les numéros au-dessus des marqueurs et amélioration de la lisibilité du parcours.	2h
06.05.2026	Entretien avec l'Expert 2	Rencontre et entretien technique avec M. Mveng (Expert 2). Présentation de l'état d'avancement actuel du projet et démonstration des fonctionnalités développées. Contrôle complet des travaux effectués. Validation de l'avance prise sur le planning initial.	0.5h

		Discussion sur les attentes concernant la présentation finale du projet (structure et contenu).	
07.05.2026	Implémentation de la sauvegarde	Développement de la fonction de sauvegarde automatique. Les points, couleurs, commentaires et heures sont désormais enregistrés dans le navigateur de l'utilisateur. (RouteStorage.vue)	1.5h
07.05.2026	Développement de la fonction LoadRoute	Création d'un bouton de chargement dans l'interface. Implémentation de la logique pour récupérer les données sauvegardées et redessiner les marqueurs et les lignes sur la carte lors du clic.	1.5h
07.05.2026	Sprint Review (Fin de sprint)	Analyse des objectifs fixés en début de semaine. Vérification de la complétion de toutes les fonctionnalités de base (points, couleurs, commentaires, polylines, LocalStorage).	0.5h
07.05.2026	Rétrospective de sprint	Analyse du déroulement de la phase de code. Identification des succès (respect du planning) et des difficultés (gestion des événements Vue.js).	0.5h
11.05.2026	Configuration de l'environnement pour les tests	Installation et configuration de l'outil Vitest pour les tests unitaires. Mise en place de l'infrastructure de test pour le projet Vue.js.	1h
11.05.2026	Développement des tests unitaires	Rédaction des tests automatisés avec IA (Gemini et Chatgpt) pour composant (MapContainer.vue). Vérification du rendu correct et de la logique interne.	3h
11.05.2026	Développement des tests unitaires	Rédaction des tests automatisés avec IA (Gemini et Chatgpt) pour composant (Etapeltem.vue, RouteStorage.vue, Sidebar.vue). Vérification du rendu correct et de la logique interne.	3.5h
12.05.2026	Préparation du plan de tests manuels	Rédaction d'une liste de points de contrôle (checklist) pour vérifier chaque fonctionnalité (ajout de points, modification des couleurs, sauvegarde).	3h
12.05.2026	Exécution des tests manuels et validation	Test complet de l'application. Vérification du comportement de la carte et du stockage. Résultat : l'application est conforme au cahier des charges.	1h
12.05.2026	Documentation	Avancement de la documentation technique (dont 1h réalisée en mobilité lors du trajet en train).	3h

13.05.2026	Amélioration technique (Géolocalisation)	Analyse et correction du problème de précision (accuracy) sur Desktop. Amélioration de la fonction findMyLocation pour gérer la précision (accuracy). Ajout d'une logique de vérification : affichage de la marge d'erreur uniquement si elle est > 150m.	3h
13.05.2026	Gestion des erreurs (UX)	Mise en place d'alertes spécifiques pour les environnements à basse précision (Desktop/IP > 500m) afin d'améliorer la transparence vis-à-vis de l'utilisateur.	1h
13.05.2026	Documentation technique (README.md)	Rédaction du fichier README.md professionnel incluant le guide d'installation, la stratégie de test et l'architecture des données (MLD).	3h
18.05.2026	Rédaction de la conclusion	Rédaction finale des conclusions du projet : analyse des objectifs atteints (Cahier des charges), définition des points positifs (Vitest, documentation) et négatifs (limites de la polyline/routing).	3h
18.05.2026	Analyse critique et difficultés	Documentation détaillée de la gestion de la précision GPS et des choix techniques effectués pour optimiser l'UX face aux limitations des navigateurs.	2h
18.05.2026	Déploiement sur Vercel	Configuration finale du projet pour la production. Importation du dépôt GitHub sur Vercel et déploiement réussi de l'application. URL publique générée : https://monparcours-neon.vercel.app/	2h
19.05.2026	Finalisation du rapport	(Planification initiale vs réelle), restructuration des conclusions et mise en page globale.	2h
19.05.2026	Révision du code et ajout de commentaires		3h
20.05.2026	Contrôles finaux et rendu du rapport	Relecture ultime du document, vérification des liens (GitHub et Vercel), génération de la table des matières automatique. Exportation finale au format PDF	3h
Nombre total d'heures			90h

5.4 Manuel d'installation

L'ensemble de la procédure pour installer et lancer le projet localement est disponible dans le fichier README de mon dépôt GitHub.

Vous pouvez y accéder via le lien suivant :

https://github.com/akrblt/TPI_Trip_Planner/tree/main/monparcours#readme

Installation & Setup

1 Install Dependencies

To install all required packages, run:

```
# Clone the project
git clone [https://github.com/akrblt/TPI_Trip_Planner/tree/main]

# Enter the directory
cd monparcours

# Install dependencies
npm install
```

2 Run for Development

To start the local development server with **Hot Module Replacement (HMR)**:

```
# Run the dev server
npm run dev
[!NOTE]
The app will be available at: http://localhost:5173
```

3 Build for Production

To generate a highly optimized, production-ready build in the dist/ folder:

```
# Build the project
npm run build
```

Testing Strategy

The project implements a robust testing strategy to ensure application stability:

Unit Testing: Powered by **Vitest**, focusing on core logic such as ID generation and data structure validation.

Component Testing: Ensuring the Map and Sidebar components react correctly to data changes.

Manual Testing: Full validation of the Geolocation Accuracy logic across different environments (GPS simulation via DevTools vs WiFi/IP).

To execute the test suite:

```
npm run test
```

5.5 Manuel d'utilisation

L'application Mon Parcours a été conçue pour être intuitive et simple d'utilisation. Voici les étapes pour planifier un itinéraire :

1. Ouverture de l'application (Initialisation) : Lors du premier lancement, la carte s'initialise et se centre automatiquement par défaut sur la ville de Yverdon-les-Bains (centre de référence de l'application).

2. Détermination de la position (My Position) : Lorsque l'utilisateur clique sur le bouton "My Position" situé dans l'interface :

- L'application demande l'autorisation d'accéder aux coordonnées géographiques.
- Une fois l'accès accordé, la carte effectue un zoom dynamique et se déplace instantanément vers la position actuelle de l'utilisateur.
- *Gestion de la précision* : Si la marge d'erreur détectée est $> 150\text{m}$ (comme c'est souvent le cas sur les ordinateurs fixes via l'IP), la valeur technique s'affiche. Si elle dépasse 500m , un message d'avertissement conseille d'activer le GPS.

3. Ajout d'étapes : Cliquez directement sur n'importe quel endroit de la carte pour y placer un marqueur.

4. Personnalisation d'une étape : En cliquant sur un marqueur ou via la barre latérale (Sidebar), vous pouvez :

- Ajouter un commentaire (ex : "Hôtel", "Visite musée").
- Définir une heure d'arrivée prévue.
- Assigner une couleur spécifique au marqueur pour catégoriser vos étapes.

5. Visualisation de l'itinéraire : Contrairement à un tracé automatique, l'application laisse le contrôle total à l'utilisateur pour générer l'itinéraire ;

- Une fois que vous avez ajouté et personnalisé vos différentes étapes sur la carte, cliquez sur le bouton "Draw Lines" situé dans l'interface.

- Cette action déclenche l'algorithme qui relie dynamiquement tous les marqueurs entre eux par une ligne (polyline), respectant rigoureusement l'ordre chronologique des étapes définies.

6. Sauvegarde et Chargement (Save / Load) : L'application intègre un système de persistance des données localisé, entièrement contrôlé par l'utilisateur ;

- **Sauvegarde (Button Save) :** Une fois que votre itinéraire est configuré et que les lignes sont tracées, cliquez sur le bouton "Save". Cette action enregistre instantanément l'intégralité de vos étapes (noms, commentaires, heures, couleurs, ordre) dans le LocalStorage de votre navigateur.

- **Chargement (Button Load) :** Si vous rafraîchissez la page ou fermez le navigateur, l'application se réinitialise sur Yverdon-les-Bains. Pour récupérer votre travail, il vous suffit de cliquer sur le bouton "Load" dans la barre latérale : l'application restaure alors immédiatement votre dernier itinéraire complet sur la carte.

5.6 Archives du projet

rapport_tpi_mon_parcours.pdf : Ce document constitue le dossier complet et unique du projet. Il regroupe l'intégralité des éléments produits durant le TPI, organisés comme suit :

Le Résumé du TPI (5.1) : Synthèse d'une page des objectifs et résultats.

Le Journal de travail (5.3) : Suivi chronologique détaillé des tâches accomplies chaque jour.

Le Manuel d'Installation (5.4) : Procédure technique pour la mise en place de l'environnement, l'installation des dépendances (npm install) et la configuration de la clé API.

Le Manuel d'Utilisation (5.5) : Guide pas à pas pour lancer l'application et utiliser ses fonctionnalités.